

CYTALK

TOPIC - BIT MANIPULATION AND BASICS OF BITMASKING

Goal

- Understand bit manipulation and basic operators.
- Learn tips for bit manipulation.
- Common patterns in bit manipulation problems.
- Practice some problems

WHERE IS BIT MANIPULATION USED

Problems may or may not directly involve bit manipulation.

Some problems use bitwise operators in the problem statement itself, whereas some problems might use bit manipulation indirectly.

For example, if the problem involves powers of 2, it might be related to binary. Similarly, powers of other numbers might be in their own base

binary. Similarly, powers of other numbers might be in their own base.

How to approach Bit Manipulation problems?

“When solving a bitwise problem, think in bits”

When solving a problem that uses addition, multiplication, etc. we use decimal, which is the appropriate base for us.

When solving a problem that uses a bitwise operator, we need to use binary, which is the appropriate base.

Let’s see some basic bitwise operators we’ll use throughout.

Bitwise Operators - OR

The OR operator (“|”) takes every corresponding bit of the two numbers and checks whether at least one of them is set.

A	B	Result
1	1	1
1	0	1
0	1	1
0	0	0

Bitwise Operators - AND

The AND operator (“&”) takes every corresponding bit of the two numbers and checks whether both of them is set

A	B	Result
1	1	1
1	0	0
0	1	0
0	0	0

Bitwise Operators - XOR

The XOR operator (“^”) takes every corresponding bit of the two numbers and checks whether exactly one of them is set.

A	B	Result
1	1	0
1	0	1
0	1	1
0	0	0

Bitwise Left Shift (“<<”)

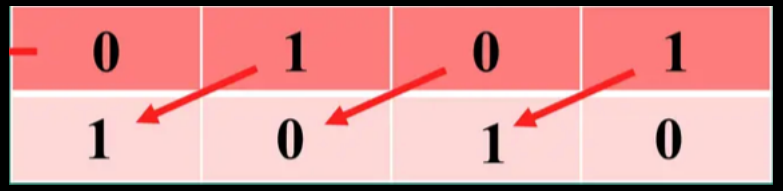
$A << B$ shifts the A to the left by B bits, adding B zeros at the end.

It is same as A x 2^B

For example,

5 in binary = 0101

5 << 1



After this operation it becomes

1010 which is 10 in decimal.

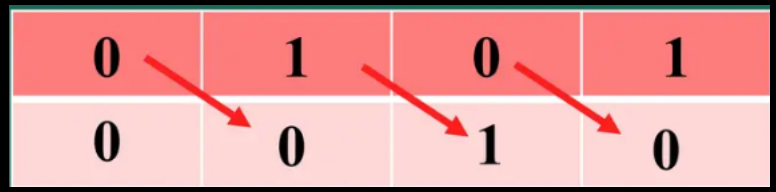
Bitwise Right Shift (“>>”)

$A >> B$ shifts the A to the right by B bits, deleting B zeros from the end. This value is the same as floor of $A \div 2^B$.

For example,

5 in binary = 0101

5 >> 1



After this operation it becomes

0010_which is 2 in decimal.

Bitwise NOT operator / complement operator. (“ ~ “)

- It flips all the bits of the a number.
- 0 -> 1 and 1 -> 0;

- For example

- N = 5 (“00000101”); ~(N) = (“11111010”);

Summary

A = 5 (“0101”) B = 9 (“1001”)

Operators	Meaning	Example
&	Perform bitwise AND operation.	(A&B) = 1 i.e. 0001
|	Perform bitwise OR operation.	(A|B) = 13 i.e. 1101
^	Perform bitwise XOR operation.	(A^B) = 12 i.e. 1100
~	Perform bitwise NOT operation.	~A = 10 i.e. 1010
<<	Binary Left shift operator shifts all bits towards left by certain number of specified bits. It is denoted by <<.	A<<1 = 1010 i.e. 10
>>	Binary Right shift operator shifts all bits towards right by certain number of specified bits.	A>>1 = 0010 i.e. 2

Questions :

Problem : XORwice

(https://codeforces.com/problemset/problem/1421/A)



WHY BITWISE?

Why Are Bitwise Operations Faster Than `/`, `*`, and `%`?

Bitwise operations (`<<`, `>>`, `&`, `|`, `^`) are faster than normal arithmetic operations like division (`/`), multiplication (`*`), and modulo (`%`) because they operate directly at the binary level and require fewer CPU cycles.

- **Multiplication** (`*`) involves multiple additions and bit shifts internally.
- **Division** (`/`) **and Modulo** (`%`) are significantly more complex as they require iterative subtraction, shifting, and carry propagation.

On many processors:

- **Multiplication takes ~3-5 CPU cycles**
- **Division takes ~10-30 CPU cycles**
- **Modulo takes about the same as division (~10-30 cycles)**
- **Bitwise operations (`&`, `|`, `^`, `<<`, `>>`) take just 1 CPU cycle** because they directly manipulate bits.

To check the parity of the number

- We can simply AND the number with 1 to check if the number is ODD or EVEN.
- It is much faster than checking (x % 2 == 0) or not.

```
void main(){
  int x;
  cin >> x;

  // lets take x = 5 for now;
  // to check if its odd or even we can just see
  if( x & 1 ){
    cout << "odd num" << endl;
  }
  else{
    cout << "even num" << endl;
  }
}
```

To check if the number is a power of 2 or not

O(1) operation to see if the number is a power or 2.

For example, Lets take X = 8 (“1000”);
(we can also use this to unset the last set bit);
If we take bitwise AND of X with X-1 that is

X = 1000

& X - 1 = 0111

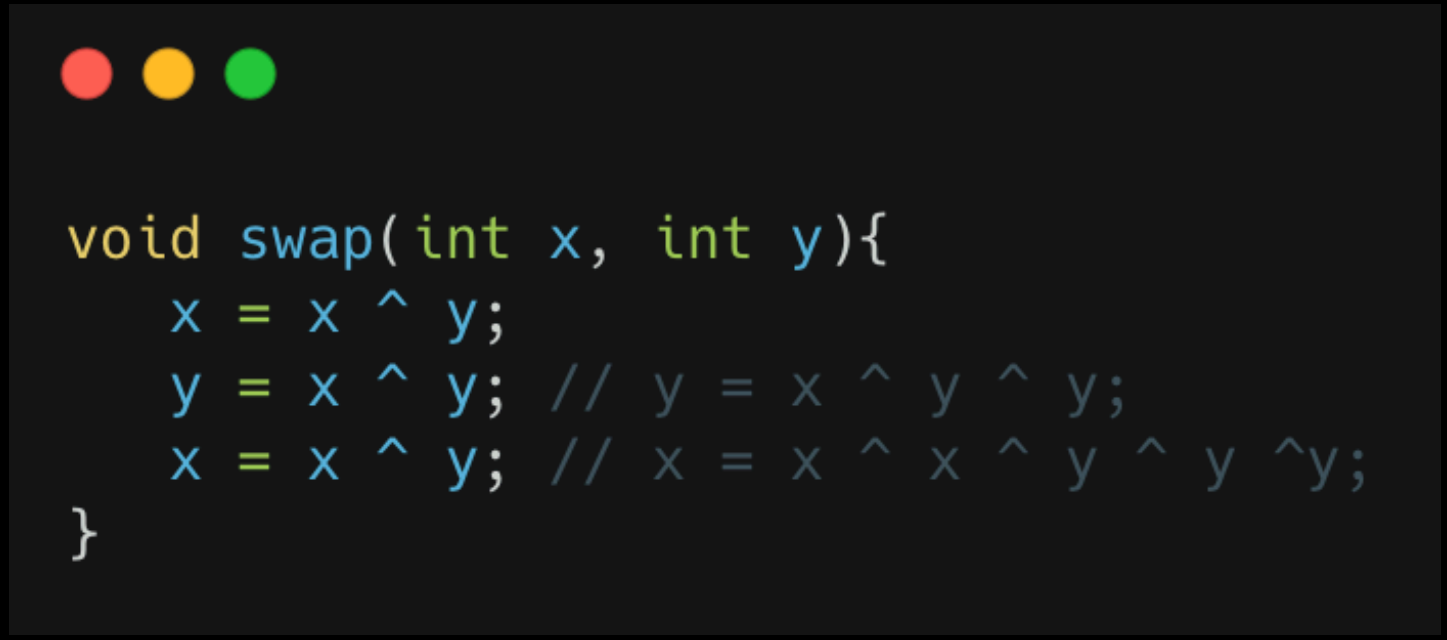
0000

```
void solve(int n){
    if(n == 0 || n & (n-1) != 0){
      cout << "not a power of 2" << endl;
    }
   else if( n & (n-1)) {
     cout << "it is power of 2" << endl;
   }
}
```

- Practice Question - (https://codeforces.com/problemset/problem/1475/A)

Swapping two number (using bitwise operator)

- We can easily swap two numbers without using a new temporary variable.



```
void swap(int x, int y){
    x = x ^ y;
    y = x ^ y; // y = x ^ y ^ y;
    x = x ^ y; // x = x ^ x ^ y ^ y ^y;
}
```

Finding out x % 2^k

- Lets say we have a number x. The remainder when we divide the number by any power of 2.

- $X \% 2^k = x \& ((1 << k) - 1);$

Taking example 10 % 4;

Two important identities

$$A + B = (A \wedge B) + 2 * (A \,\&\, B)$$

$$A + B = (A \mid B) + (A \,\&\, B)$$

- Let’ s Take,

  A = 9 (“ 1001 “) == 8 + 1

  B = 3 (“ 0011 “) == 2 + 1

  A ^ B = 1010 = 10 = 8 + 2; // obtain all the different bits

  A & B = 0001 = 1; // obtain all the same bits which we also
  need to add in both numbers that why
  multiply by 2

A I B = 1011 ; // obtain all different bits but also same bits has been added
One time, so we need to add the same bits one more time

https://codeforces.com/contest/2085/problem/C

Playing with the k’th bit (1 << k) == 2^K

1. Check if Kth bit is set
2. Toggle the Kth bit
3. Set the Kth bit
4. Unset the Kth bit

Checking the Kth bit

X & (1 << k)

We have X = 11 (“1011”);

Lets say we have to check 2nd bit, we take k = 2, (1 << k) is equivalent to “0100”

X & (1 << k) == 1011 & 0100 == 0000

Toggle the Kth bit

X ^ (1 << k)

Toggling a bit means flipping it:

0 → 1 1 → 0

This can be done using the XOR (^) operator with a bit mask

To toggle the `k`th bit of a number `n`: $n \oplus (1 \ll k)$

- `1 << k` creates a mask where only the `k`th bit is set to `1`.
- XORing (`^`) with this mask flips the `k`th bit.

```
n      = 1101  (13 in binary)
mask   = 0010  (1 << 1)
Result = 1111  (15 in binary)
```

```cpp
#include <iostream>
using namespace std;

int toggleKthBit(int n, int k) {
    return n ^ (1 << k);
}

int main() {
    int n = 13, k = 1;
    cout << "After toggling " << k << "th bit: " << toggleKthBit(n, k) << endl;
    return 0;
}
```

Setting the Kth Bit

`X | (1 << k)`

- Setting a bit means ensuring it's `1`, regardless of its previous state.
- Done using the **OR** (`|`) operator.

- `1 << k` creates a mask with only the `k`th bit set to `1`.
- ORing (`|`) with this mask ensures the `k`th bit is `1`.

```
n      = 1001  (9 in binary)
mask   = 0010  (1 << 1)
Result = 1011  (11 in binary)
```

```cpp
#include <iostream>
using namespace std;

int setKthBit(int n, int k) {
    return n | (1 << k);
}

int main() {
    int n = 9, k = 1;
    cout << "After setting " << k << "th bit: " << setKthBit(n, k) << endl;
    return 0;
}
```

Unsetting the Kth bit

`X & ~(1 << k)`

- Unsetting a bit means ensuring it's `0`, regardless of its previous state.
- Done using the **AND (`&`)** operator with a negated bit mask.

- `1 << k` creates a mask with only the `k`th bit set to `1`.
- `~(1 << k)` inverts the mask, setting all bits to `1` except the `k`th bit.
- ANDing (`&`) with this ensures the `k`th bit is `0`.

```
n      = 1011  (11 in binary)
mask   = 0010  (1 << 1)
~mask  = 1101  (~(1 << 1))
Result = 1001  (9 in binary)
```

```cpp
#include <iostream>
using namespace std;

int unsetKthBit(int n, int k) {
    return n & ~(1 << k);
}

int main() {
    int n = 11, k = 1;
    cout << "After unsetting " << k << "th bit: " << unsetKthBit(n, k) << endl;
    return 0;
}
```

UNDERSTANDING BITSET

Definition:

A `bitset` in C++ is a fixed-size sequence of bits (0s and 1s) that provides efficient storage and manipulation of binary data. It is implemented as a specialized data structure that allows bit-level operations such as setting, resetting, flipping, and checking individual bits.

Why use `bitset` over `vector<int>`?

- `Bitset` is memory-efficient as it stores each bit separately, whereas `vector<int>` typically uses 32 or 64 bits per integer.
- It allows direct bitwise operations (`&`, `|`, `^`, `~`, `<<`, `>>`), making it faster for certain computations.
- Unlike `vector<int>`, a `bitset` provides built-in functions such as `.count()` (number of set bits), `.any()`, `.none()`, and `.to_string()`.
- Faster access to individual bits due to its optimized internal representation.

```cpp
#include <bitset>
#include <iostream>
using namespace std;

int main() {
    bitset<8> b(5); // 00000101
    cout << "Bitset: " << b << endl;
    return 0;
}
```

Output

```
Bitset: 00000101
```

Some Builtin functions of bitset

1. Setting a bit (b.set( ) , b.set (bit))

-> Sets a bit at a given position to 1.

```
bitset<5> a("10101");
a.set(1);
cout << a << endl;
```

Output : 10111

2. Reset a bit (b.reset(), b.reset(bit))

Sets a bit at a given position to 0.

```
a.reset(1);
cout << a << endl;
```

Output : 10101

- Flip bits (b.flip( ))

-> Toggles all bits.

```
a.flip( );
cout << a << endl;
```

Output :

```
01010
```

- Converting Bitset to integer (to_ulong)

  Converts bitset to unsigned long.

```
bitset<4> arr(8);
int r = (int) arr.to_ulong();
cout << r << endl;
```

Output : 8

- Convert Bitset to String (to_string)

```
string s = arr.to_string();
cout << s << endl;
```

Output : 1000

- Checking if a bit is set (a.test(bit))

  Returns true if a specific bit is 1.

```
bitset<5> a("1000");
cout << a.test(3) << endl;
```

Output : 1

- Counting number of set bits (a.count( ))

```
bitset<11> s("10101010011");
cout << s.count( ) << endl;
```

Output : 6

- Checking if any bit is set or none of the bit is set (a.any( ) , a.none( ))

```
bitset<5> a(9);

bitset<6> b(0);

cout << a.any( ) << " " << b.any( ) << endl;

cout << a.none( ) << " " << b.none( ) << endl;
```

Output :

```
1 0
0 1
```

- Checking If all the bits are set (a.all( ))

```
bitset<3> a("111");
cout << a.all( ) << endl;
```

Output :

```
1
```

Some functions we will use in Bit Manipulation

Decimal to binary conversion

Output :

```
156 in Binary is 10011100
```

```cpp
#include <iostream>
using namespace std;

void decimalToBinary(int n){
    int arr[11], i = 0, num = n;

    // Until the value of n becomes 0.
    while(n != 0){
        arr[i] = n % 2;
        i++;
        n = n / 2;
    }
    cout << num << " in Binary is ";

    // Printing the array in Reversed Order.
    for(i = i - 1; i >= 0;i--){
        cout << arr[i];
    }
    cout << endl;
}
int main() {
    int n = 156;
    decimalToBinary(n);

    return 0;
}
```

Counting number of set bits in a number

Output : 4

```cpp
#include <iostream>

using namespace std;

int SetBits(int n){

   int count = 0;

   while(n > 0){

      count += n&1;

      n >>= 1;

   }

   return count;

}

int main)( ){

   int n = 23;

   cout << SetBits(n) << endl;

}
```

above 1200 and 1300 problems

Now some builtin functions

`__builtin_popcount(x)`	Returns the number of set bits (`1` s) in `x` .
`__builtin_ffs(x)`	Finds the index (1-based) of the first (rightmost) set bit in `x` .
`__builtin_clz(x)`	Counts the number of leading zeros before the first `1` bit in `x` .
`__builtin_ctz(x)`	Counts the number of trailing zeros after the last `1` bit in `x` .
`__builtin_parity(x)`	Returns `1` if the number of set bits is odd, otherwise `0` .

```cpp
#include <iostream>
using namespace std;

int main() {
    unsigned int x = 300; // Binary: 0b00000000'00000000'00000001'00101100
    cout << "__builtin_popcount(x) = " << __builtin_popcount(x) << endl;
    cout << "__builtin_ffs(x) = " << __builtin_ffs(x) << endl;
    cout << "__builtin_clz(x) = " << __builtin_clz(x) << endl;
    cout << "__builtin_ctz(x) = " << __builtin_ctz(x) << endl;
    cout << "__builtin_parity(x) = " << __builtin_parity(x) << endl;
    return 0;
}
```

Output :

```
__builtin_popcount(x) = 4
__builtin_ffs(x) = 3
__builtin_clz(x) = 23
__builtin_ctz(x) = 2
__builtin_parity(x) = 0
```

Some tricks

x & ~((1 << i+1) – 1)	Clears all bits of x from LSB to ith bit
x & ((1 << i) – 1)	Clears all bits of x from MSB to ith bit
ch | ' '	Upper case English alphabet ch to lower case
ch & '_'	Lower case English alphabet ch to upper case

Questions :

Problem 1: Raising Bacteria

(https://codeforces.com/problemset/problem/579/A)



Questions :

Problem 2: And then there were K

(https://codeforces.com/problemset/problem/1527/A)



Questions :

Problem 3: Fedor and new game

(https://codeforces.com/contest/467/problem/B)



Questions :

Problem 4: AND 0 , Sum Big

(https://codeforces.com/contest/1514/problem/B)



Questions :

Problem 5: Little girl and maximum XOR

(https://codeforces.com/problemset/problem/276/D)



Questions :

Problem 6: Trip to the olympiad

(https://codeforces.com/problemset/problem/2057/C)



Questions :

Problem 7: AND, OR and Square sum

(https://codeforces.com/contest/1368/problem/D)



Questions :

Problem 8: Peter and Combination lock

(https://codeforces.com/problemset/problem/1097/B)



Questions :

Problem 9: Rock and Lever

(https://codeforces.com/contest/1420/problem/B)



Questions :

Problem 10: Gardner and the array

(https://codeforces.com/problemset/problem/1775/B)





Thanks For Joining